

1 Description et déroulement du projet

Ce projet va consister à construire un Mini Système d'Exploitation très simple. Le projet se fera en équipe et en partageant le travail entre vous. Il faudra bien modéliser en premier lieu le projet afin de pouvoir répartir les différentes tâches à effectuer entre vous.

Dans ce projet, il est important que vous mettiez en évidence vos compétences en système d'exploitation, en modélisation et savoir en maîtrise de son temps afin de fournir un produit fonctionnel.

Le sujet de ce projet est un cadre minimal à mettre en place mais il est possible **si et seulement si** celui-ci est fonctionnel, de faire preuve de créativité et de développer tout ce qui vous paraîtra intéressant. Il devra être codé en langage C et il faudra veiller à structurer le code en séparant les fonctions, les commandes shell et la structure du SGF dans des fichiers différents et à commenter les fonctions du projet.

Il vous est demandé de réaliser :

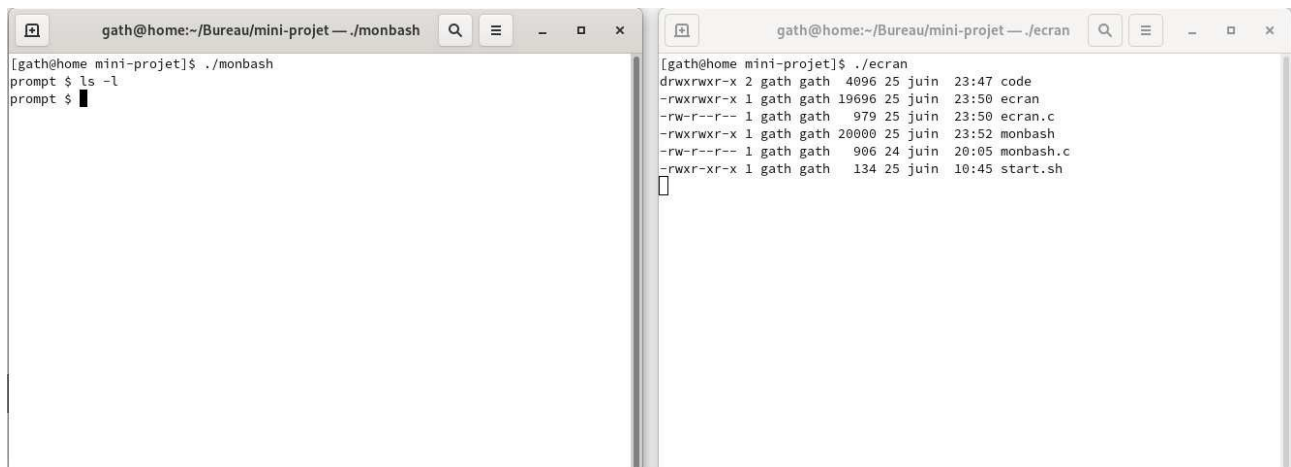
- Un petit rapport décrivant l'analyse du ou des besoins de l'utilisateur, définissant le système à réaliser et comment vous pensez le réaliser, donnant la description globale des fonctions principales, la ou les structures de données prévues, la liste des fonctions principales et enfin la répartition des tâches entre les membres de l'équipe.
- Un rapport et vos programmes sources (pas d'exécutable) seront à déposer sur Moodle.

2 Le projet

Remarques générales : une bibliographie/sitographie est donnée à titre indicatif à la fin de ce document.

2.1 Cahier des charges

Ce projet va consister à réaliser un émulateur de gestionnaire de fichiers (SGF) avec un interpréteur bash. Le but de ce projet est donc de créer votre propre mini-interpréteur de commandes shell en utilisant des fonctions qui interagissent avec un SGF. Lorsqu'on exécute le projet, on doit pouvoir avoir accès à une fenêtre qui attend des commandes (comme le shell de linux) et une fenêtre qui affiche le résultat des instructions.



```
[gath@home mini-projet]$ ./monbash
prompt $ ls -l
prompt $
```

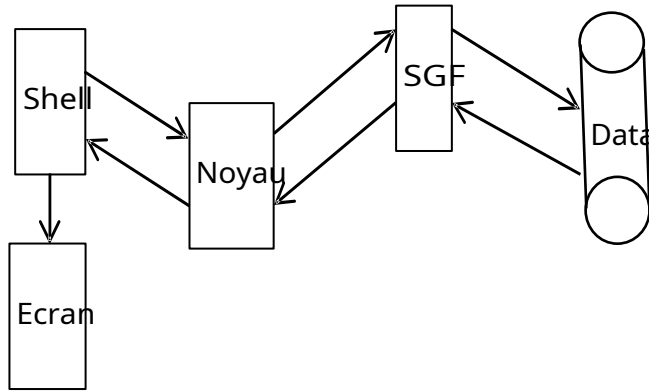
```
[gath@home mini-projet]$ ./ecran
drwxrwxr-x 2 gath gath 4096 25 juin 23:47 code
-rwxrwxr-x 1 gath gath 19696 25 juin 23:50 ecran
-rw-r--r-- 1 gath gath 979 25 juin 23:50 ecran.c
-rwxrwxr-x 1 gath gath 20000 25 juin 23:52 monbash
-rw-r--r-- 1 gath gath 906 24 juin 20:05 monbash.c
-rwxr-xr-x 1 gath gath 134 25 juin 10:45 start.sh
```

Les notions de droits peuvent être appréhender a posteriori. On se cantonnera à « utilisateur » et « les autres ».

Il est donc demandé dans ce mini-projet de :

- Faire un interpréteur basique (mini bash) ;
- Créer une structure interne des inodes et l'arborescence des fichiers ; (UTC502)
- Définir un ensemble de fonctions qui permettent d'interagir avec le SGF ;
- Définir un ensemble de commandes shell qui utilisent ces fonctions.

Dans le schéma ci-dessous, un utilisateur va lancer un terminal puis exécute des commandes. Chaque commande va faire appel à des fonctions du noyau permettant d'interroger/modifier le SGF qui lui-même utilise un fichier de données pour stocker les informations.



2.2 Le Shell

Le shell est un programme (processus) qui va s'exécuter et se mettre en attente d'une commande. Ce processus est chargé de créer un processus fils (noyau). Le shell va envoyer la commande au noyau et ce dernier va utiliser la bibliothèque de fonctions du SGF afin de traiter la commande et renvoyer les résultats vers l'écran.

2.3 L'Ecran

L'écran est un terminal classique (ce n'est pas un fils du shell) dans lequel un programme en C s'exécute et attends des données à afficher.

2.4 Les communications

Une réflexion sur les moyens de communication dans ce système est à faire. Quels moyens de communication peuvent être utilisé pour communiquer entre le processus Shell et son fils le processus Noyau ? Même question pour la communication entre le processus Shell et le processus Écran. Il peut être intéressant de montrer que vous avez plusieurs cordes à votre arc.

2.5 Système de fichiers

Le but serait d'imaginer un système de fichier qui s'inspire du SGF de Linux. On se contentera d'un seul niveau d'indirection.

Afin de simuler un disque dur (sur le disque dur physique de votre machine), il est possible de créer un gros fichier sur le disque qui va avoir un fonctionnement s'approchant du fonctionnement du SGF Linux (c'est par exemple ce qui se passe dans une machine virtuelle - il y a un gros fichier image sur le disque dur).

Dans ce gros fichier, il faut réfléchir à certains points : comment peut-on accéder aux différents fichiers, comment les créer, comment les détruire, comment les placer (contiguë ? par bloc ? chaîné ? <-- si on détruit un fichier comment récupère-t-on la place) ?

Suivant les décisions prises concernant ce fichier, il faut créer des primitives qui permettent d'y accéder (par exemple myopen permet d'ouvrir le fichier (on cherche où il est), myread permet de lire son contenu, mycreat permet de le créer, ...

Mais on peut également utiliser de la mémoire avec ces mêmes fonctions

2.6 Fonctions SGF

Les fonctions de manipulation des fichiers pourraient être :

- `_mycreat(nom,mode)`: crée un fichier et retourne son inode
- `_myopen(nom,mode)`: ouvre/crée un fichier et retourne son inode
- `_myclose(inode)`: ferme un fichier
- `_myread(inode,buffer,nombre)`: lit nombre octets mis dans buffer
- `_mywrite(inode,buffer,nombre)`: écrit nombre octets pris dans buffer

Les fonctions des répertoires/fichiers pourraient être :

- `_mkdir(nom)`: créer un répertoire
- `_rmdir(nom)`: supprime un répertoire vide
- `_unlink(nom)`: efface une entrée du répertoire

2.7 Interpréteur shell et langage de commande

La surcouche Shell devrait pouvoir permettre à l'utilisateur d'effectuer les commandes suivantes :

- Commandes sur des répertoires : `ls`, `mkdir`, `rmdir`, `cd`
- Prendre en compte quelques arguments comme par exemple : `ls -l`
- Commandes sur des fichiers : `cp`, `rm`, `mv`, `cat`, `echo "texte" > file`
- Autres commandes : `df` qui permettra d'avoir des infos sur votre SGF

2.8 Bibliographie/sitographie

Bash + SGF

http://langevin.univ-tln.fr/cours/UPS/extra/chapitre3_sgf.pdf

<https://cours.polymtl.ca/inf2610/documentation/notes/chap11.pdf>

<http://lipn.univ-paris13.fr/~cerin/SE/SF1.pdf>

Exemples

<https://github.com/brenns10/lsh>